

DAY 25 ASSIGNMENT

SOFTWARE SUPPLY CHAIN FAILURES

Name:- Rishabh Raj

1. What is a software supply chain?

A software supply chain is the complete process and set of people, tools, code, libraries, dependencies, build systems, repositories, and deployment processes used to create and deliver software. Security problems in any part of this chain can affect the final application.

2. What is a software dependency?

A software dependency is an external package, library, framework, module, or component that an application relies on to perform some function. Dependencies save development time but can introduce security and maintenance risks.

3. Explain a direct dependency.

A direct dependency is a package that the application explicitly includes in its project configuration. For example, if a project directly installs a logging library and imports it in its code, that library is a direct dependency.

4. Explain a transitive dependency.

A transitive dependency is a dependency that is installed because another dependency requires it. The application may not directly list or use it, but it can still become part of the software and therefore needs security monitoring.

5. Why can outdated components create security risks?

Outdated components may contain known vulnerabilities that have already been fixed in newer versions. Attackers can use publicly known vulnerabilities in old versions to compromise applications, steal information, or execute unauthorized actions.

6. Why can unmaintained components be a concern?

An unmaintained component may no longer receive security fixes, bug fixes, or compatibility updates. If a new vulnerability is discovered, there may be no reliable patch, making continued use risky.

7. Why should software come from trusted sources?

Trusted sources reduce the risk of downloading modified, malicious, fake, or compromised packages. Using official repositories, verified publishers, package signatures, checksums, and controlled internal registries helps establish software integrity and provenance.

8. Why should unused dependencies be removed?

Unused dependencies increase the software attack surface and create unnecessary maintenance work. Even if a package is not actively used, it may contain a vulnerability or malicious code and may be pulled into builds. Removing unnecessary dependencies reduces risk.

9. What does SBOM stand for?

SBOM stands for Software Bill of Materials.

10. Explain an SBOM in your own words.

An SBOM is an inventory or list of the components used to build a software product. It can include direct and transitive dependencies, versions, suppliers, licenses, and component identifiers. It helps an organization understand what is inside its software.

11. Give three types of information an SBOM can help an organization understand.

An SBOM can help identify: (1) component names and versions, (2) direct and transitive dependencies and their relationships, and (3) known vulnerabilities or security advisories associated with components. It can also provide supplier and license information.

12. Does an SBOM automatically fix vulnerabilities? Explain.

No. An SBOM does not automatically fix vulnerabilities. It provides visibility into the software components so security teams can identify affected components, assess risk, and decide whether to patch, upgrade, replace, or otherwise mitigate them.

13. What is a CVE?

CVE stands for Common Vulnerabilities and Exposures. A CVE is a publicly recognized identifier assigned to a specific cybersecurity vulnerability. CVE information helps organizations consistently track and discuss known vulnerabilities.

14. Why should organizations monitor security advisories?

Security advisories provide information about newly discovered vulnerabilities, affected versions, available patches, and recommended mitigations. Monitoring them helps organizations respond quickly before known weaknesses are exploited.

15. What is patch management?

Patch management is the process of identifying, evaluating, testing, approving, deploying, and verifying software updates and security patches. Good patch management keeps systems and dependencies protected against known vulnerabilities.

16. Why should updates be tested before production deployment?

Updates should be tested to make sure they do not break application functionality, introduce compatibility problems, or cause unexpected behavior. Testing allows organizations to deploy security fixes safely while maintaining availability and reliability.

17. What is CI/CD at a beginner level?

CI/CD means Continuous Integration and Continuous Delivery or Deployment. CI automatically builds and tests code changes as developers integrate them. CD automates the process of preparing and delivering tested software to staging or production environments.

18. Why is CI/CD part of the software supply chain?

CI/CD systems build, test, package, sign, and deploy software. Because they handle source code, dependencies, artifacts, and deployment credentials, a compromise of the pipeline can affect the integrity and security of the software delivered to users.

19. Explain Least Privilege for a CI/CD system.

Least Privilege means giving CI/CD jobs, runners, service accounts, and deployment credentials only the permissions required for their specific tasks. For example, a build job should not automatically have unrestricted access to production databases or all cloud resources.

20. Why should production secrets not be stored openly in source-code repositories?

Source-code repositories may be copied, forked, logged, cached, or accessed by many people. If production passwords, API keys, tokens, or certificates are committed, attackers may obtain them and gain unauthorized access. Secrets should instead be stored in a secure secret-management system and rotated if exposed.